DATA 260 - Homework 1

Student Name: Priyank Mehta

SJSU ID: 019152562

GitHub Repository: https://github.com/priyankmehta5/data260-2562

1. Personal Configuration

This report documents the design, implementation, deployment, and evaluation of the DATA-260 Homework 1 system. The assigned domain was calculated using the last four digits of my SJSU Student ID.

My SID4 is 2562, producing a DOMAIN_ID of 2.

The application uses port 8762, calculated as 8000 + (2562 modulo 900).

Therefore, the assigned domain for this project is **Municipal Transit Incidents**.

Configuration value	Result
SID4	2562
PORT_BASE	8762
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2
Assigned domain	Municipal transit incidents
Processor	Intel(R) Core(TM) i7-8750H CPU @ 2.20 GHz
Memory	15.86 GB RAM
Operating system	Windows 10
Dedicated GPU	NVIDIA GeForce GTX 1050 Ti
Integrated GPU	Intel(R) UHD Graphics 630
Local model	qwen3:1.7b
Implementation Commit	da28c955e4a11c21d367707d6735c829890e715e

2. Part 1 - Transit Incident Web Application

2.1 Domain Schema

The assigned entity is a municipal transit incident. Before creating the application, I defined the entity fields, data types, validation requirements, and category values in DOMAIN_SCHEMA.md.

The entity contains the following fields:

Field	Type	Required	Description
incidentTitle	String	Yes	Short title identifying the incident
transitRoute	String	Yes	Transit route affected by the incident
submitterEmail	Email	Yes	Email address of the submitter
incidentDescription	String	Yes	Detailed incident description
incidentCategory	String	Yes	Classification of the incident
termsAccepted	Boolean	Yes	Records acceptance of the terms
submissionDate	Datetime	Generated	Time of successful submission

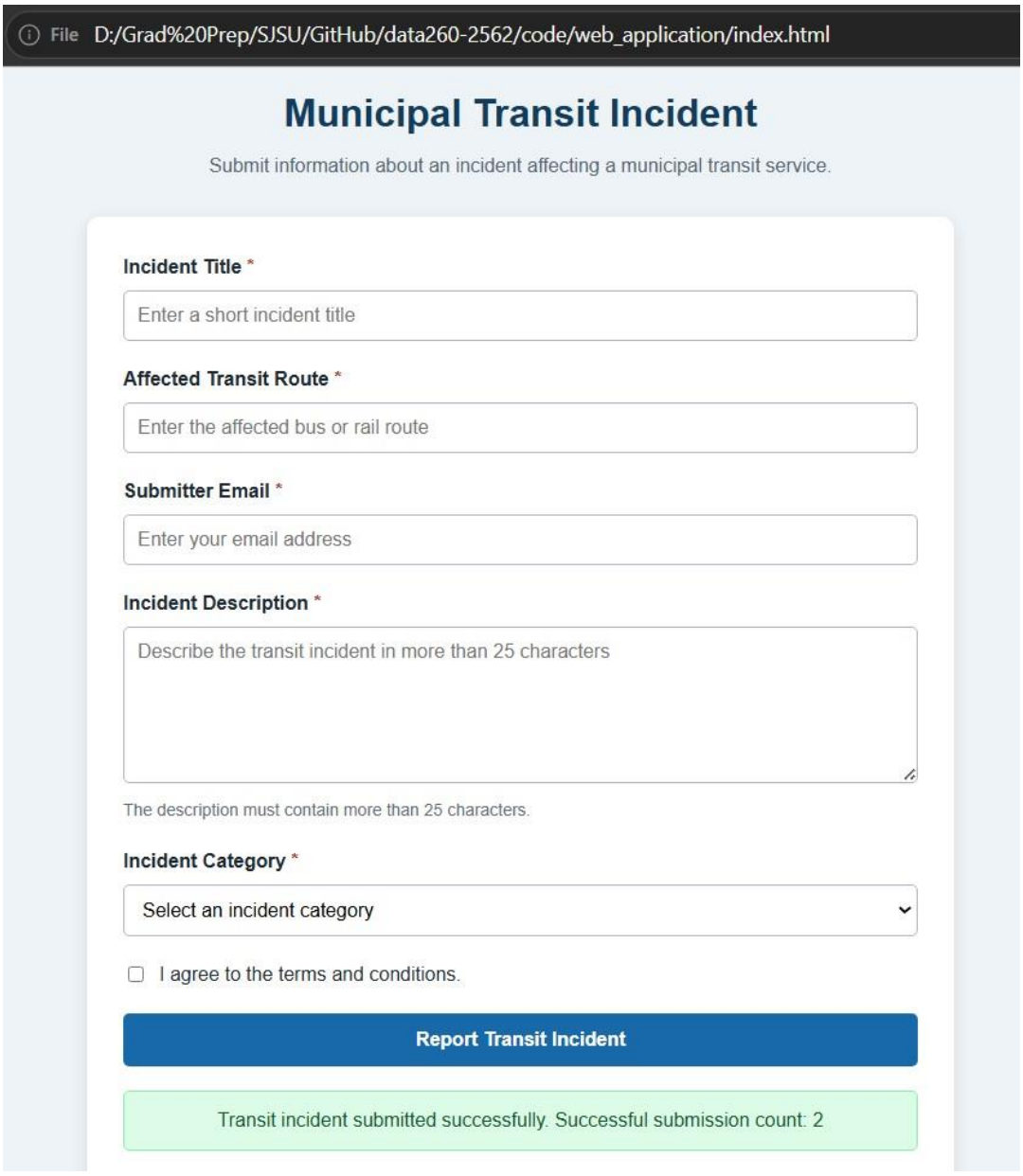
The four category values are Delay, Service Disruption, Safety Incident, and Infrastructure Issue. The incident description must contain more than 25 characters, and the terms checkbox must be selected before submission.

2.2 HTML Form

The page title is HW1-Priyank Mehta, and the largest heading identifies the entity as Municipal Transit Incident. The form contains the required primary field, secondary field, email address, description, category selection, terms checkbox, and submit button.

<title>HW1-Priyank Mehta</title> <h1>Municipal Transit Incident</h1> <form id="incidentForm"> <input type="text" id="incidentTitle" name="incidentTitle" placeholder="Enter a short incident title" required autofocus > <input type="text" id="transitRoute" name="transitRoute" placeholder="Enter the affected bus or rail route" required > <input type="email" id="submitterEmail" name="submitterEmail" placeholder="Enter your email address" required > <textarea id="incidentDescription"

name="incidentDescription" placeholder="Describe the transit incident in more than 25 characters" required ></textarea> <select id="incidentCategory" name="incidentCategory" required > <option value="" disabled selected> Select an incident category </option> <option value="Delay">Delay</option> <option value="Service Disruption"> Service Disruption </option> <option value="Safety Incident"> Safety Incident </option> <option value="Infrastructure Issue"> Infrastructure Issue </option> </select> <input type="checkbox" id="termsAccepted" name="termsAccepted" > <label for="termsAccepted"> I agree to the terms and conditions. </label> <button type="submit"> Report Transit Incident </button> </form> <script src="script.js"></script>

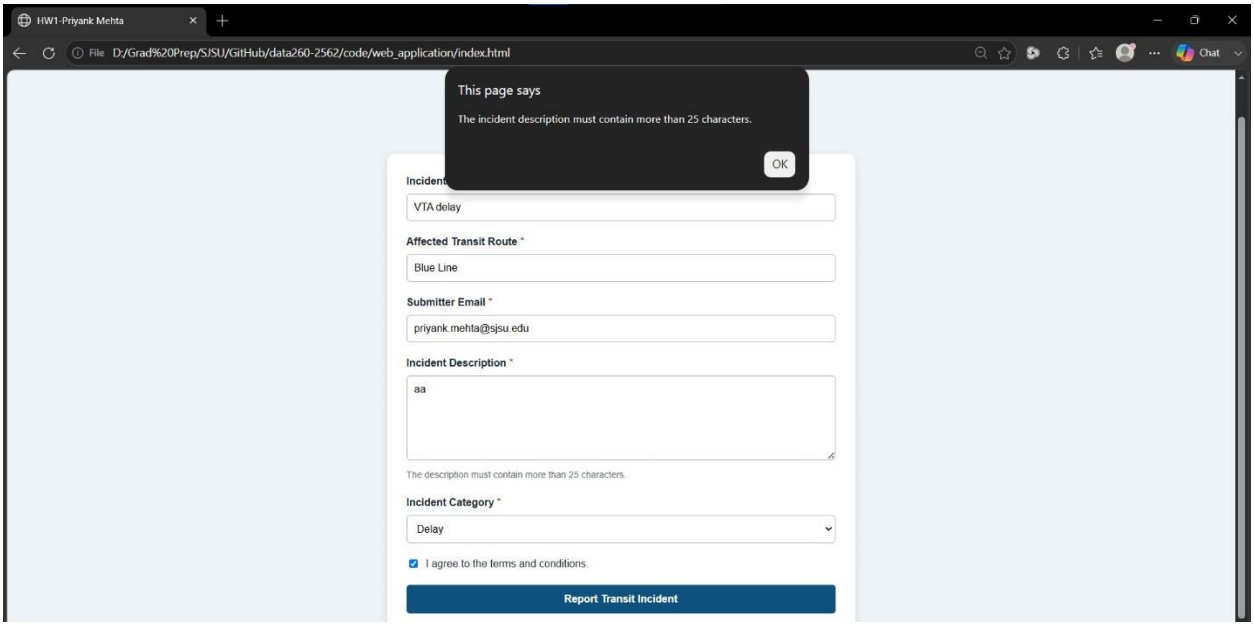


Municipal transit incident form with the required fields and four domain-specific categories.

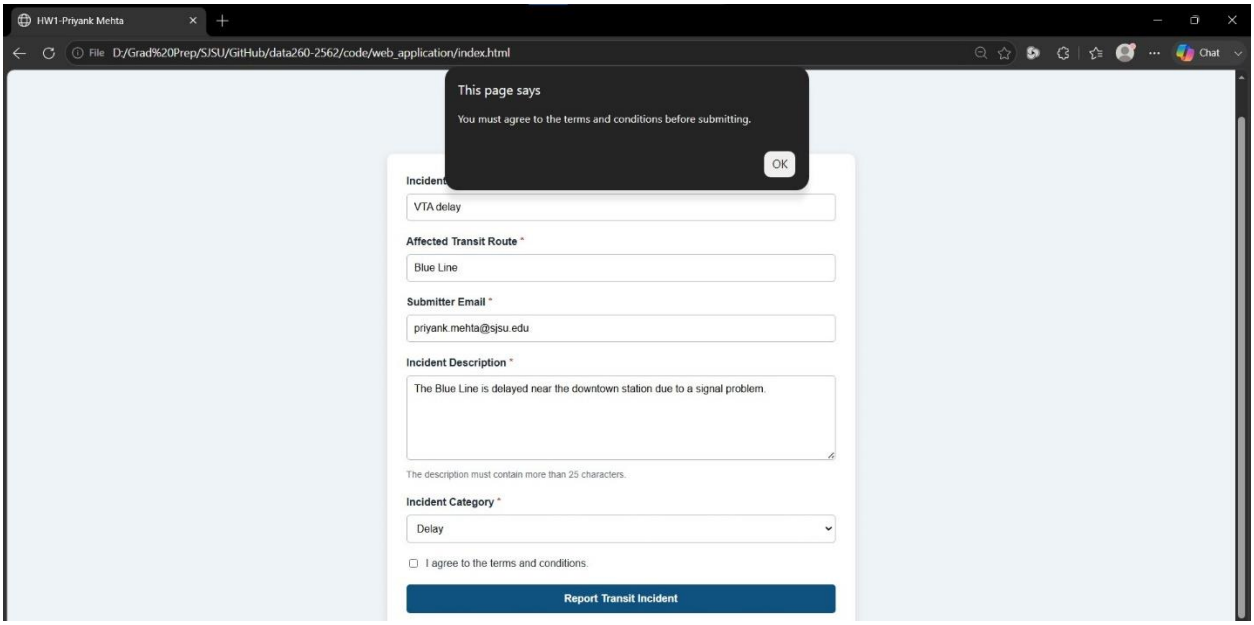
2.3 JavaScript Validation and Processing

I created an arrow function to validate the incident description and terms checkbox. A description containing 25 or fewer characters is rejected. The function also rejects the submission if the terms checkbox is not selected.

```
const validateForm = () => { const incidentDescription = document .getElementById("incidentDescription") .value .trim(); const termsAccepted = document .getElementById("termsAccepted") .checked; if (incidentDescription.length <= 25) { alert( "The incident description must contain more than 25 characters." ); document.getElementById("incidentDescription").focus(); return false; } if (!termsAccepted) { alert( "You must agree to the terms and conditions before submitting." ); document.getElementById("termsAccepted").focus(); return false; } return true; };
```



Validation alert displayed when the incident description does not contain more than 25 characters.



Validation alert displayed when the terms and conditions checkbox is not selected.

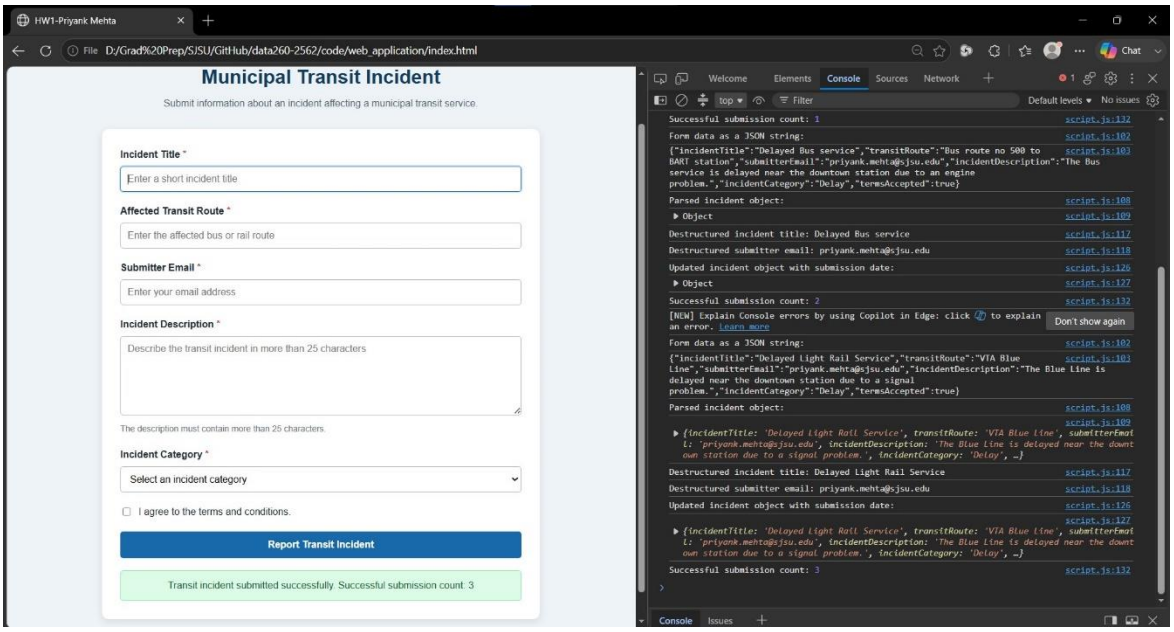
After successful validation, the entered values are stored in an object and converted to a JSON string. I parsed the string back into an object and use object destructuring to extract the incident title and submitter email.

The spread operator creates an updated object containing the original fields and a generated submissionDate. A closure maintains the number of successful submissions.

```
const createSubmissionCounter = () => {
    let count = 0;

    return () => {
        count += 1;
        return count;
    };
};

const trackSuccessfulSubmission = createSubmissionCounter();
```

```
const incidentData = {
    incidentTitle,
    transitRoute,
    submitterEmail,
    incidentDescription,
    incidentCategory,
    termsAccepted
};

const incidentJsonString = JSON.stringify(incidentData);
console.log("Form data as a JSON string:");
console.log(incidentJsonString);

const parsedIncidentData = JSON.parse(incidentJsonString);

const {
    incidentTitle: extractedIncidentTitle,
    submitterEmail: extractedSubmitterEmail
} = parsedIncidentData;

console.log(
    "Destructured incident title:",
    extractedIncidentTitle
);
console.log(
    "Destructured submitter email:",
    extractedSubmitterEmail
```

```
);

const updatedIncidentData = {
    ...parsedIncidentData,
    submissionDate: new Date().toISOString()
};

console.log(
    "Updated incident object with submission date:"
);
console.log(updatedIncidentData);

const submissionCount = trackSuccessfulSubmission();
console.log("Successful submission count:", submissionCount);
```



Browser console showing the JSON string, destructured values, updated object with the submission date, and successful submission count.

The output confirms that the form data was converted to JSON correctly. It also shows that object destructuring, the spread operator, and the closure worked as required.

2.4 Local Docker Deployment

I used Nginx Alpine to package and serve the static web application.

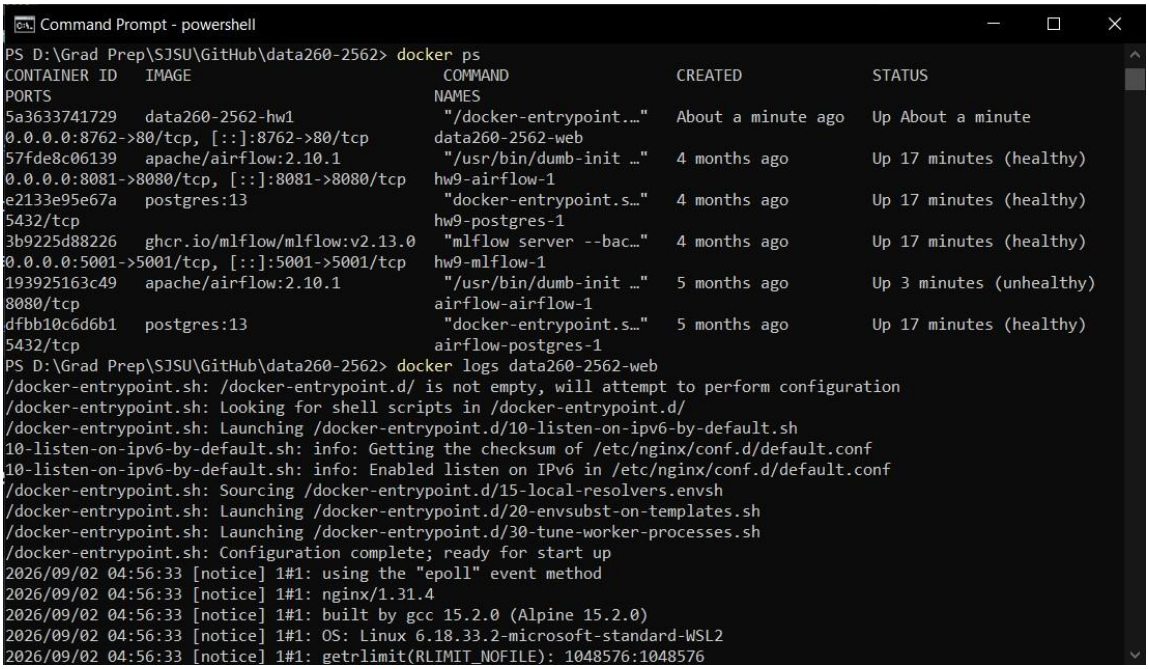
```
FROM nginx:alpine

RUN rm -rf /usr/share/nginx/html/*

COPY . /usr/share/nginx/html/

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

The Docker image was built using:

```
docker build -f code/Dockerfile -t data260-2562-hw1 code/web_application
```

The container was started by mapping local port 8762 to container port 80:

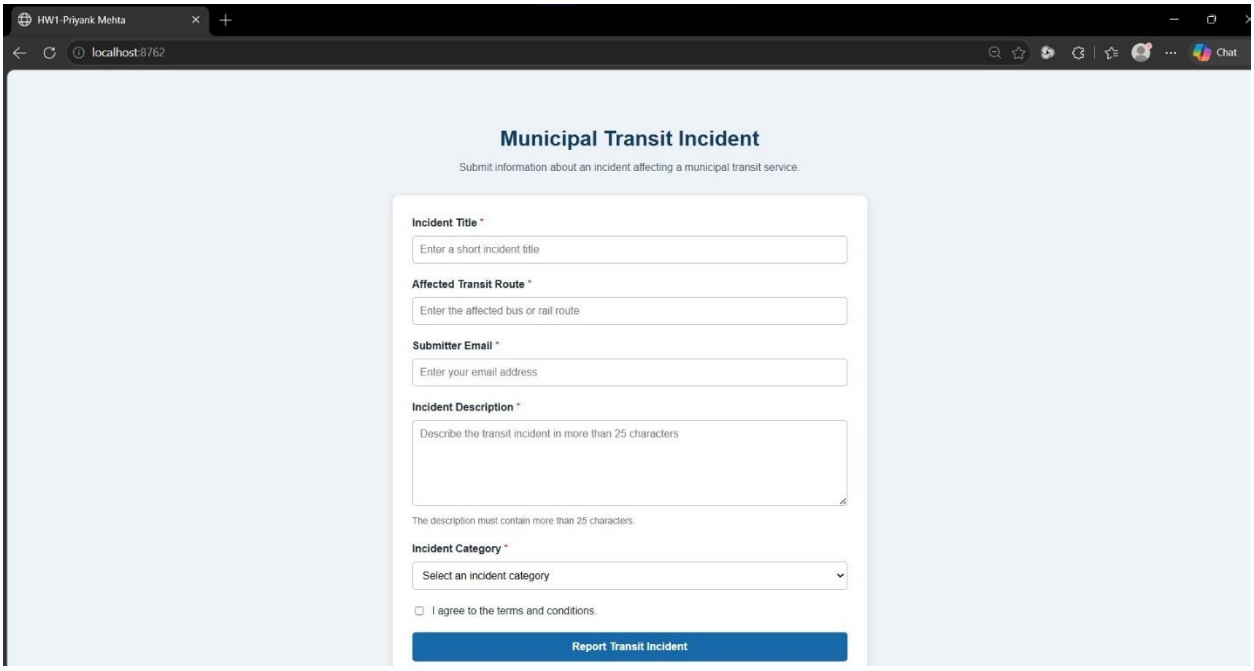
```
docker run -d -p 8762:80 --name data260-2562-web data260-2562-hw1
```



Docker container running with local port 8762 mapped to Nginx port 80.

The application was tested at:

http://localhost:8762

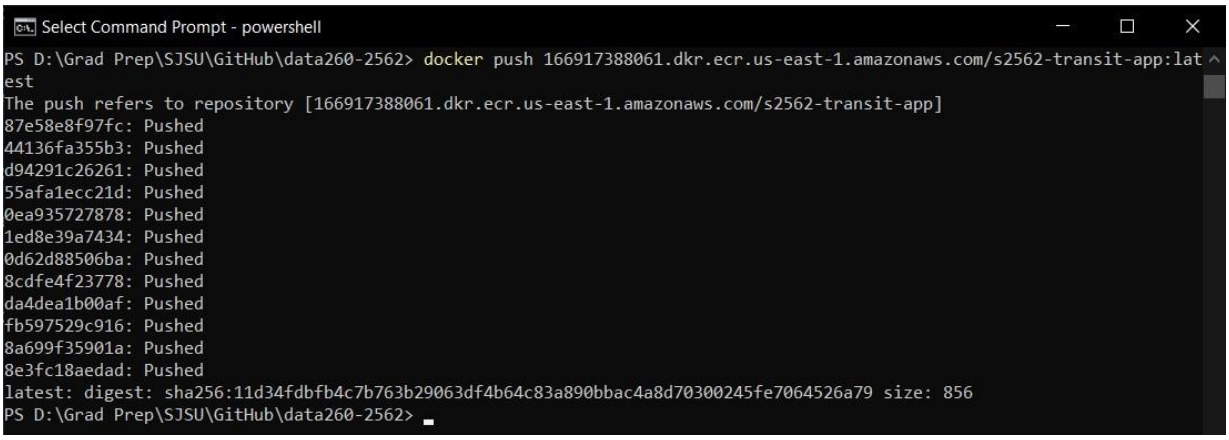


Transit incident application running locally from the Docker container on port 8762.

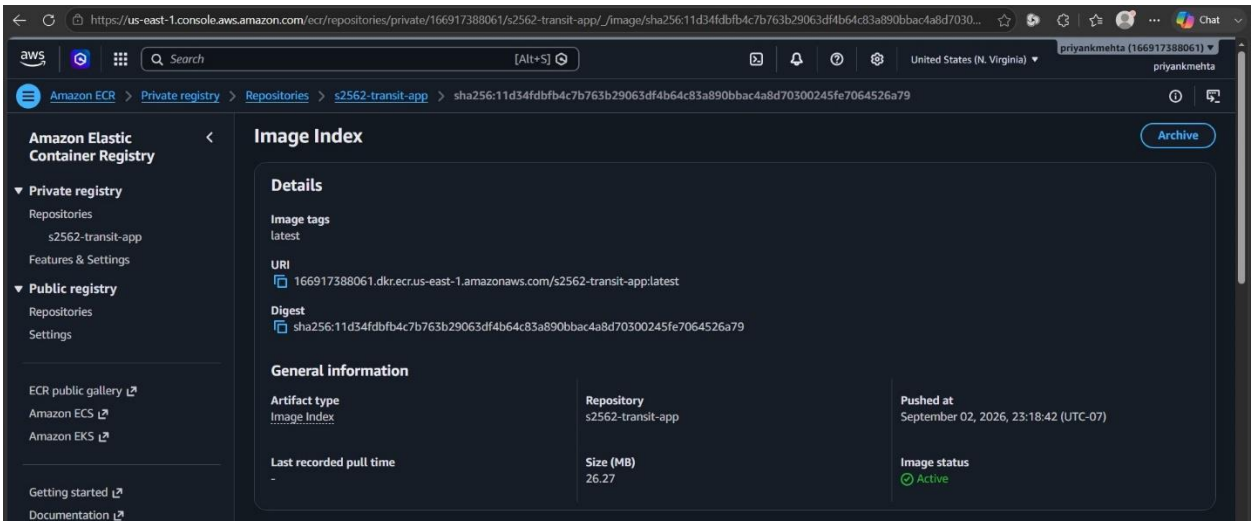
2.5 AWS ECS Deployment

I created a private Amazon ECR repository named s2562-transit-app. The local Docker image was tagged and pushed to the repository in the us-east-1 region.

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 166917388061.dkr.ecr.us-east-1.amazonaws.com docker tag data260-2562-hw1:latest 166917388061.dkr.ecr.us-east-1.amazonaws.com/s2562-transit-app:latest docker push 166917388061.dkr.ecr.us-east-1.amazonaws.com/s2562-transit-app:latest

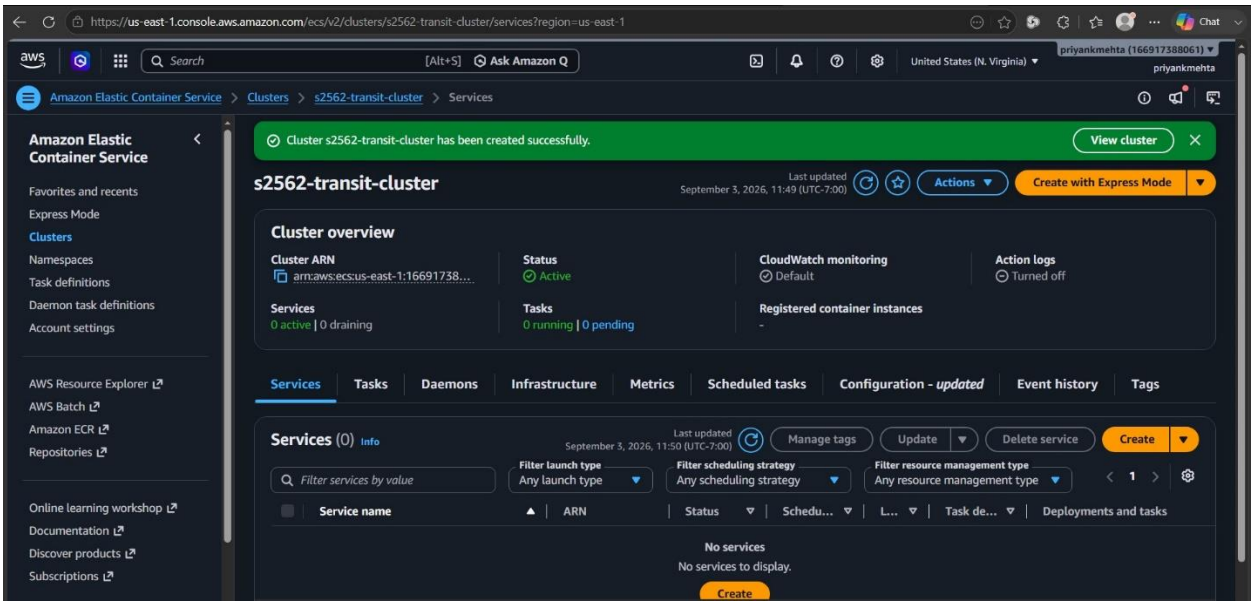


Successful Docker image push to the private Amazon ECR repository.

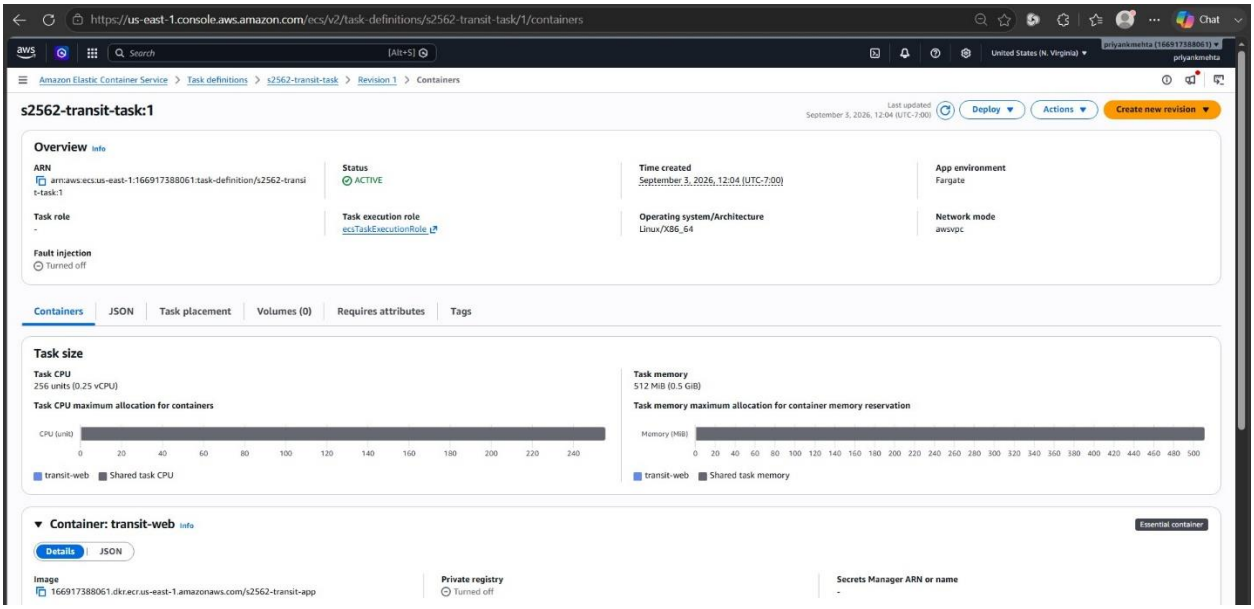


Amazon ECR image with the latest tag and recorded image digest.

I created the ECS cluster s2562-transit-cluster. The task definition used AWS Fargate with 256 CPU units, 512 MB of memory, awsvpc network mode, and container port 80.

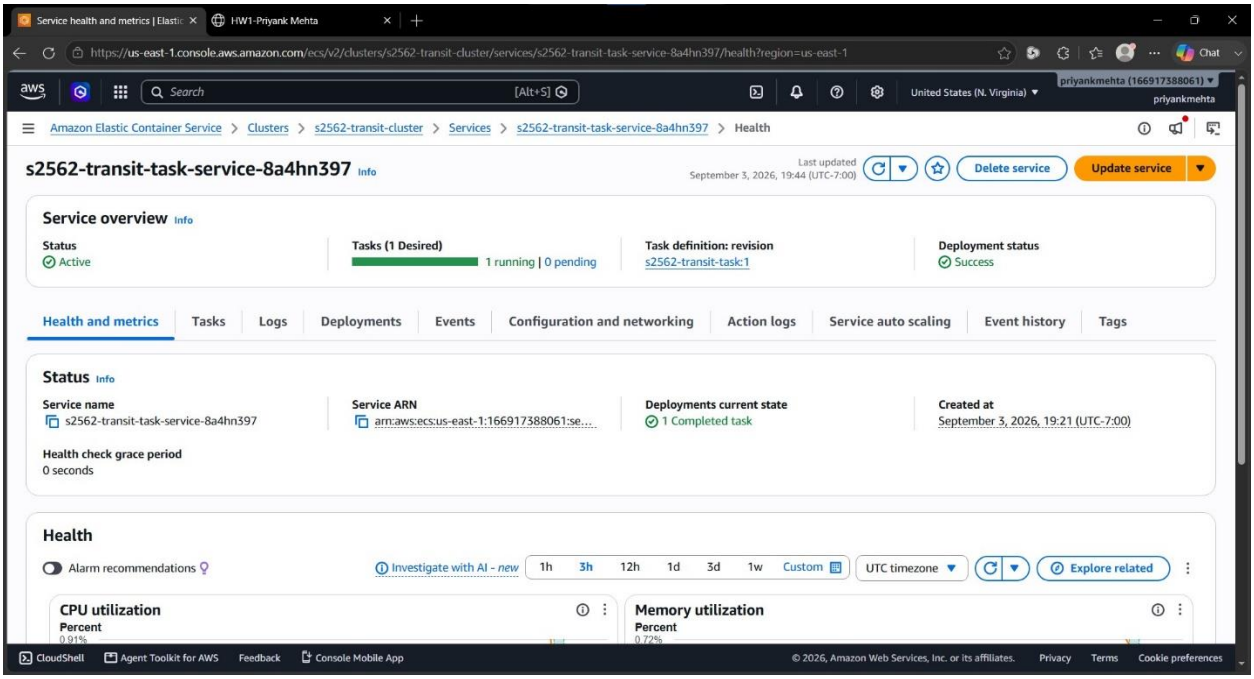


Active Amazon ECS cluster created for the transit application.

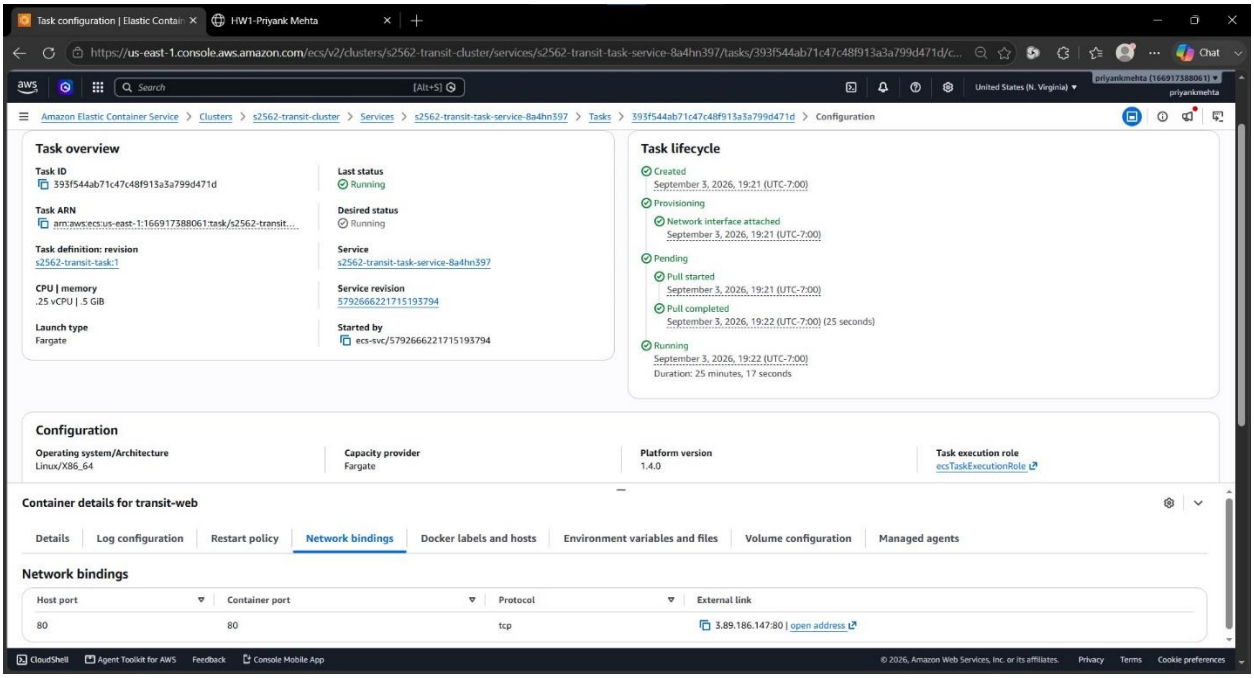


ECS task definition configured with the ECR image and container port 80.

The ECS service was configured with one desired task. The service reached the active state with one running task and zero pending tasks.

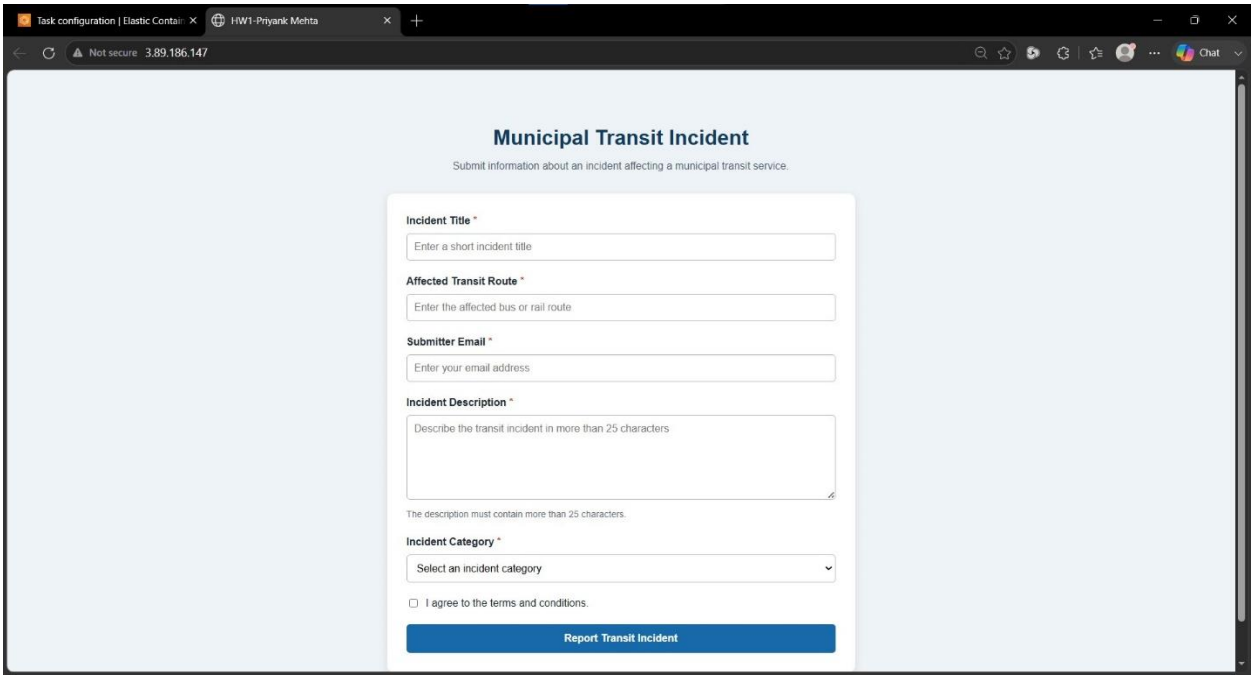


ECS service showing one desired task and one running task.



AWS Fargate task in the running state.

Finally, I opened the assigned public IP address in a browser and confirmed that the containerized transit incident application was accessible through AWS.



Transit incident application successfully running through the AWS public IP address.

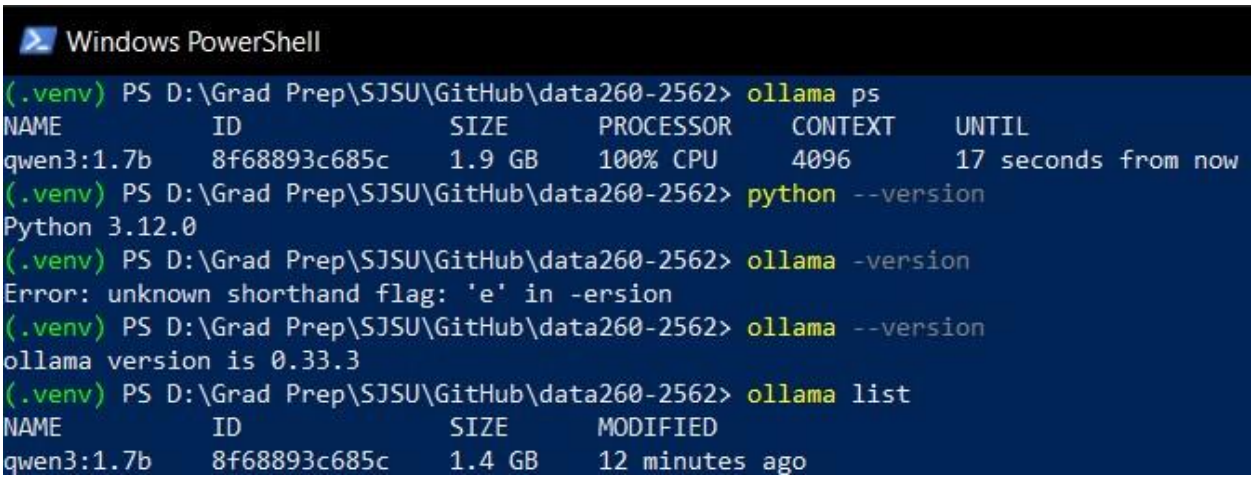
After collecting the deployment evidence, I stopped the running task and removed the temporary AWS resources to avoid unnecessary charges.

3. Part 2 - Agentic AI Workflow

3.1 Model and Environment

The agent workflow was implemented using Python and a local Ollama model. The assignment recommended qwen3:8b, but this model caused Windows to restart with a VIDEO_MEMORY_MANAGEMENT_INTERNAL stop error on my laptop.

I next tested the smaller qwen3:4b model, but it caused the same error. I therefore selected the tool-capable qwen3:1.7b model and configured Ollama for CPU-only execution. The ollama ps output confirmed that the model was using 100% CPU.



Ollama running the qwen3:1.7b model using 100% CPU.

The model was accessed through the reusable adapter in src/model_client.py. All model requests were sent through its complete(messages, tools=None) interface.

class OllamaModelClient: def __init__(self, model="qwen3:1.7b", temperature=0.0, base_url="http://localhost:11434", num_ctx=4096): self.model = ChatOllama(model=model, temperature=temperature, base_url=base_url, num_ctx=num_ctx, format="json") def complete(self, messages, tools=None): model = self.model.bind_tools(tools) if tools else self.model return model.invoke(messages)

3.2 Planner, Reviewer, and Finalizer

The workflow contains three main stages:

1. The Planner reads the incident title and description and proposes three tags and a summary.
2. The Reviewer examines the Planner output and can correct generic, repeated, or unrelated tags.
3. The deterministic Finalizer validates and publishes exactly three normalized tags and a summary containing no more than 25 words.

The domain was not hardcoded into the agent logic. The program derives the tags and summary from the title and content supplied through command-line arguments.

The main agent-call function sends a system prompt and user prompt through the model adapter. It also measures the model response time.

def call_agent(

```
    client,
    system_prompt,
    user_prompt,
    title,
    content,
    strict
):
    started = time.perf_counter()

    response = client.complete(
        [
            ("system", system_prompt),
            ("human", user_prompt)
        ]
    )

    latency_ms = round(
        (time.perf_counter() - started) * 1000
    )

    parsed = extract_json(str(response.content))
    result = coerce_reply(
        parsed,
        title,
        content,
        strict
    )
```

```
    return result, latency_ms
```

The finalization step uses the reviewed result and returns only the required tags and summary.

```
def finalize(
    planner,
    reviewer,
    title,
    content
):
    reviewed = coerce_reply(
        reviewer,
        title,
        content,
        True
    )

    if len(reviewed["data"]["tags"]) != 3:
        reviewed = coerce_reply(
            planner,
            title,
            content,
            True
        )

    return {
        "tags": reviewed["data"]["tags"],
        "summary": reviewed["data"]["summary"]
```

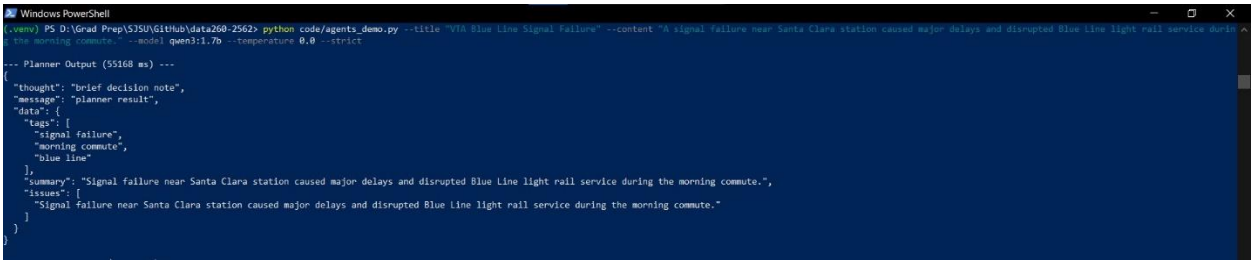
}

The exact command used was:

python code/agents_demo.py --title "VTA Blue Line Signal Failure" --content "A signal failure near Santa Clara station caused major delays and disrupted Blue Line light rail service during the morning commute." --model qwen3:1.7b --temperature 0.0 --strict

3.3 Planner Output

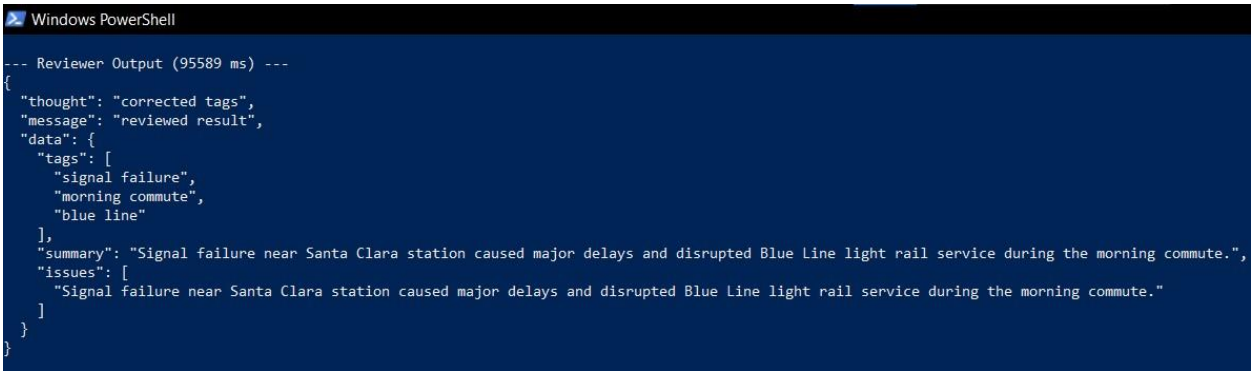
The Planner generated three domain-relevant tags and a one-sentence summary. The measured Planner latency was 55,168 milliseconds.



Planner output with three transit-incident tags, a summary, and the measured latency.

3.4 Reviewer Output

The Reviewer checked the Planner output against the original incident title and content. Its measured latency was 95,589 milliseconds.



Reviewer output after checking the Planner result.

3.5 Finalized and Publish Output

The deterministic finalization step produced valid JSON containing exactly three tags and a summary containing 20 words.

Finalized and Publish JSON output containing the validated result and Planner/Reviewer transcript.

3.6 Required Answers

Q1. **What three final tags were obtained?**

The three final tags were:

- signal failure
- morning commute
- blue line

Q2. What was the final summary?

“Signal failure near Santa Clara station caused major delays and disrupted Blue Line light rail service during the morning commute.”

The summary contains 20 words and satisfies the limit of no more than 25 words.

Q3. Did the Reviewer change anything?

No. The Reviewer returned the same three tags and summary as the Planner. Although its internal message stated that tags were corrected, the visible Planner and Reviewer results were identical.

3.7 Explanation of the Workflow

The program first receives a title and content through command-line arguments. The Planner uses this input to generate an initial structured response. The response is parsed and normalized so that invalid formatting does not reach the final output.

The Reviewer receives both the original input and the Planner response. It checks whether the tags are relevant and distinct and whether the summary meets the word limit.

Finally, the deterministic Python finalizer selects the reviewed tags and summary, enforces exactly three tags, limits the summary to 25 words, and prints the Finalized and Publish outputs as formatted JSON. This final step reduces the risk of publishing an incorrectly structured model response.

4. Part 3 - Nondeterminism Experiment

4.1 Fixed Input and Method

I used the same municipal transit incident input for every experiment run. The input was saved in reports/hw01/cases/nondeterminism_input.json before starting the experiment.

{ "title": "VTA Blue Line Signal Failure", "content": "A signal failure near Santa Clara station caused major delays and disrupted Blue Line light rail service during the morning commute." }

The agent workflow was executed 20 times with temperature 0.7 and 20 times with temperature 0.0, for a total of 40 runs. Each run recorded the temperature, final tags, final summary, and end-to-end latency.

The experiment was started with:

python code/run_nondeterminism.py --input reports/hw01/cases/nondeterminism_input.json --model qwen3:1.7b --runs 20

The main experiment loop used both required temperature values:

```
temperatures = [0.7, 0.0]


for temperature in temperatures:
    completed = sum(
        result["temperature"] == temperature
        for result in results
    )


    for run_number in range(completed + 1, runs + 1):
        result = run_pipeline(
            title,
            content,
            model,
            temperature
        )


        results.append(result)
        save_results(results)
```

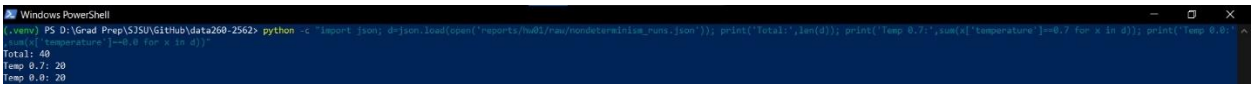
Results were saved incrementally in the following machine-readable files:

reports/hw01/raw/nondeterminism_runs.json

reports/hw01/raw/nondeterminism_runs.csv

reports/hw01/raw/nondeterminism_summary.json

The saved data was checked after completion and contained exactly 40 successful runs: 20 at each temperature.



Verification of 40 total experiment runs, including 20 runs at each temperature.

4.2 Experiment Results

Metric	Temperature 0.7	Temperature 0.0
Successful runs	20	20
Distinct tag sets	10	3
Latency p50	115,486 ms	76,804 ms
Latency p95	161,003 ms	122,841 ms
Latency p99	167,861 ms	154,367 ms

At temperature 0.7, no tag appeared in all 20 runs.

The following tags appeared in exactly one temperature 0.7 run:

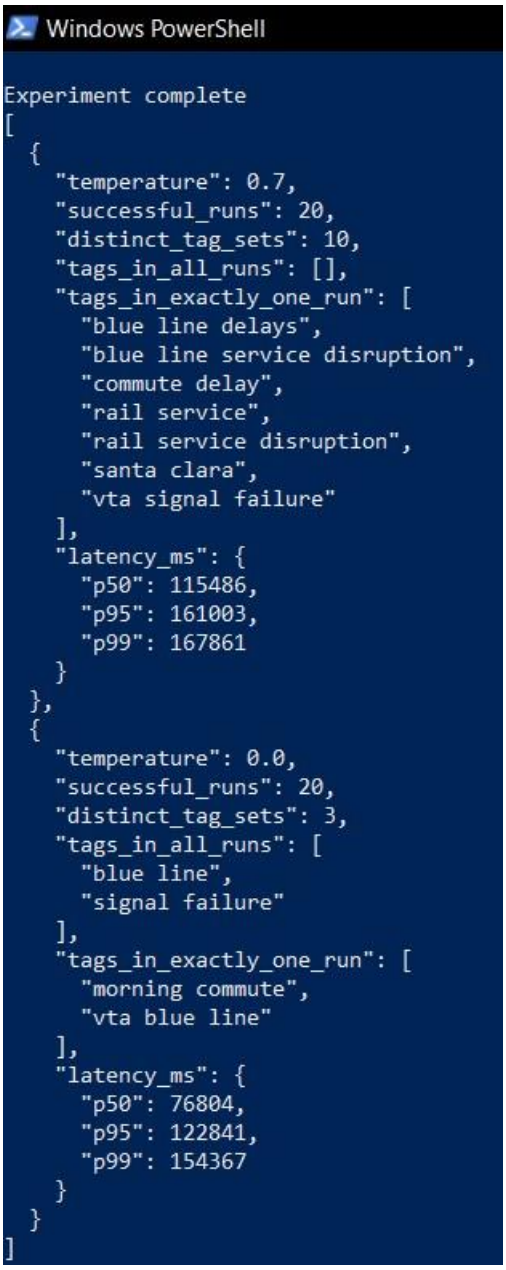
- blue line delays
- blue line service disruption
- commute delay
- rail service
- rail service disruption
- santa clara
- vta signal failure

At temperature 0.0, the following tags appeared in all 20 runs:

- blue line
- signal failure

The following tags appeared in exactly one temperature 0.0 run:

- morning commute
- vta blue line



Nondeterminism experiment summary showing tag variation and latency percentiles for both temperatures.

4.3 Interpretation

The higher temperature produced more varied output. Temperature 0.7 produced 10 distinct tag sets, while temperature 0.0 produced only 3. Therefore, reducing the temperature made the model more consistent, but it did not completely eliminate nondeterminism.

Two users submitting the identical transit incident could receive different third tags. For example, one user might receive morning commute, while another might receive santa clara

station or vta. The users could also experience different response times. At temperature 0.7, the median latency was 115,486 milliseconds, while at temperature 0.0 it was 76,804 milliseconds.

Run-to-run variation can be acceptable when tags are used for exploratory search or content discovery. Slightly different but relevant tags may provide multiple useful ways to locate a transit incident.

Variation would not be acceptable when the output controls an operational or safety-critical decision. For example, an emergency transit incident must not be classified inconsistently because different classifications could affect alerts, escalation, or the response priority.

The results show that temperature 0.0 is more appropriate when consistent tagging is important. However, deterministic validation and controlled category values would still be necessary for safety-critical use because temperature 0.0 produced three distinct tag sets rather than one.

5. Part 4 - Model Client and Token Accounting

5.1 Reusable Model Adapter

I created the reusable model adapter at the required path, src/model_client.py. The adapter provides the stable complete(messages, tools=None) interface. The Planner, Reviewer, and interactive client use this interface instead of directly invoking the model.

```
from langchain_ollama import ChatOllama




class OllamaModelClient:
    def __init__(
        self,
        model="qwen3:1.7b",
        temperature=0.0
    ):
        self.model = ChatOllama(
            model=model,
            temperature=temperature,
            reasoning=False
```

```
        )


    def complete(self, messages, tools=None):
        client = (
            self.model.bind_tools(tools)
            if tools
            else self.model
        )

        return client.invoke(messages)
```

Using an adapter separates the application from the specific model library. The implementation inside the adapter can be changed later without requiring every application file to be rewritten.

5.2 Five-Turn Conversation

The root-level AGENT.md file defines the model's behavior. It instructs the model to act as a code reviewer and return only concise bullet points.

```
You are a code reviewer.

Follow these rules for every response:

- Respond only with bullet points.

- Begin every non-empty line with `- `.

- Do not use headings, numbered lists, paragraphs, or code fences.

- Review the submitted code for correctness, clarity, security, and maintainability.

- Keep each bullet concise.
```

The client reads these instructions and includes them as the system message at the beginning of the conversation history.

```
instructions = (ROOT / "AGENT.md").read_text( encoding="utf-8" )
```

history = [{ "role": "system", "content": instructions }]

The program checks whether each non-empty response line starts with a bullet marker.

def follows_bullet_format(content):

lines =

[line.strip()

for line in content.splitlines()

if line.strip()

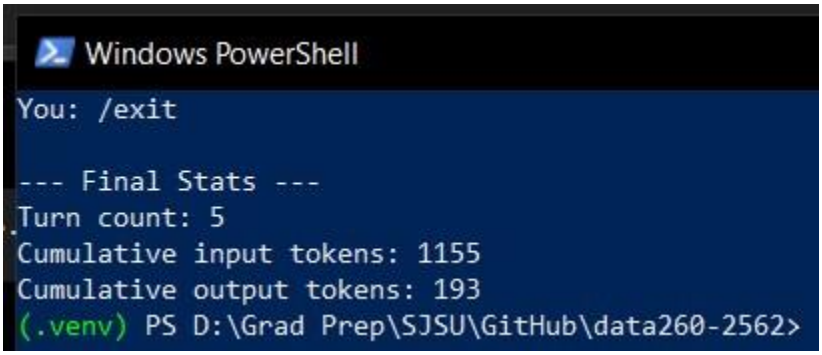
]

return bool(lines) and all(

line.startswith("- ")

for line in lines

)



Model response following the bullet-only instructions from AGENT.md.

5.3 Token Accounting

After every model response, the client reads the token information returned by Ollama. It displays the input, output, and total token counts for the current turn.

def get_token_usage(response):

usage = response.usage_metadata or {}

metadata = response.response_metadata or {}

```
input_tokens = usage.get(
"input_tokens", metadata.get("prompt_eval_count", 0)
)
output_tokens = usage.get(
"output_tokens",
metadata.get("eval_count", 0)
)
total_tokens = usage.get(
"total_tokens", input_tokens + output_tokens
)
return input_tokens, output_tokens, total_tokens
```

After a response is received, the current values are added to the cumulative totals.

```
input_tokens, output_tokens, total_tokens = (
get_token_usage(response)
)
turn_count += 1
cumulative_input += input_tokens
cumulative_output += output_tokens
print(f"Input tokens: {input_tokens}")
print(f"Output tokens: {output_tokens}")
print(f"Total tokens: {total_tokens}")
```

5.4 /stats Command

The /stats command displays the number of completed turns, cumulative token counts, and serialized conversation-history length.

```
def history_length(history):
    return len(json.dumps(history, ensure_ascii=False))
```

```
def print_stats(turn_count, cumulative_input, cumulative_output, history):
    print("\n--- Conversation Stats ---")
    print(f"Turn count: {turn_count}")
    print(f"Cumulative input tokens: {cumulative_input}")
    print(f"Cumulative output tokens: {cumulative_output}")
    print(f"History length: {history_length(history)} characters")
```

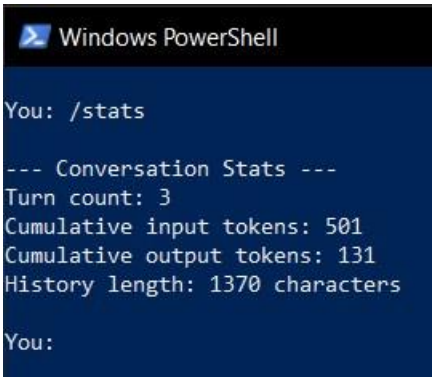
The command is handled before adding anything to the conversation history. Therefore, checking the statistics does not alter the history or consume a conversation turn.

```
if user_input == "/stats":
        print_stats(
            turn_count,
            cumulative_input,
            cumulative_output,
            history
        )
continue
```

5.5 Five-Turn Conversation Results

I ran a five-turn code-review conversation. Statistics were recorded after turn 3 and again after turn 5.

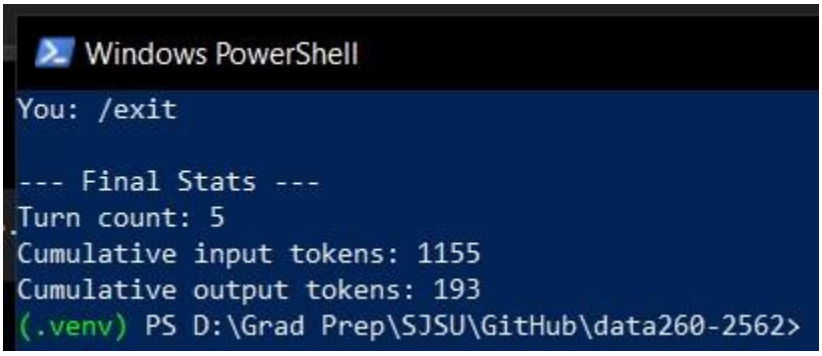
Measurement	**After Turn 3**	**After Turn 5**
Turn count	3	5
Cumulative input tokens	501	1155
Cumulative output tokens	131	193
Serialized history length	1,370 characters	1,920 characters



Conversation statistics recorded after the third model response.

Conversation statistics recorded after the fifth model response.

The final output reported five completed turns, 1,155 cumulative input tokens, and 193 cumulative output tokens.



Final cumulative token statistics displayed when the client exited.

5.6 Conversation Context Questions

Why is the prior conversation context resent with every turn?

The local model does not automatically remember earlier calls. Each request is independent, so the client resends the system instructions and previous user and assistant messages. This allows the model to understand follow-up questions and maintain continuity.

How is a system prompt different from a user message?

A system prompt defines the model's overall role, behavior, and response rules. In this application, the system prompt contains the bullet-only code-review instructions from AGENT.md. A user message contains the code or question that the user wants the model to review.

Why do input tokens grow during a conversation?

Input tokens grow because every new request includes the accumulated conversation history. As additional user prompts and model responses are stored, more text must be sent to the model during later turns. This behavior is visible in the increase from 501 cumulative input tokens after turn 3 to 1,155 after turn 5.

What eventually limits that growth?

The model's context window limits conversation growth. The context window is the maximum number of tokens the model can process in one request. When the history approaches this limit, older messages must be removed, shortened, or summarized before the conversation can continue.

6. AI Use and Troubleshooting

A complete response to the four required AI-use questions is available in reports/hw01/AI_USE.md.

7. Reproducibility and Verification

The repository includes exact setup and execution commands in:

reports/hw01/reproducible_run_instructions

These instructions explain how to:

- Create the Python virtual environment
- Install the required packages

- Build and run the Docker application
- Start Ollama in CPU-only mode
- Execute the Planner and Reviewer workflow
- Reproduce the 40-run nondeterminism experiment
- Run the five-turn model client
- Review the saved results

I also created code/verify_hw01.py as a self-check script. The verifier checked the required files, HTML title, Docker HTTP response, model-adapter interface, experiment run counts, output format, and Part 4 conversation evidence.

The verification completed with an overall status of PASS. The Docker application returned HTTP status 200, all 40 experiment runs were present, and the Part 4 five-turn statistics were found.

Homework 1 self-check completing with an overall PASS status.

The machine-readable verification output is stored in:

reports/hw01/verification.json

The console commands and timestamps for the reported results are stored in RUN_LOG.txt. The experiment results and conversation transcript are stored in reports/hw01/raw/.

8. Conclusion

This homework combined frontend development, JavaScript validation, containerization, cloud deployment, local language-model agents, nondeterminism measurement, and token accounting.

The municipal transit incident form satisfied the required HTML and JavaScript behaviors. The application ran successfully through Docker on local port 8762 and was deployed to an AWS ECS Fargate service with one running task.

The local Planner and Reviewer workflow produced exactly three tags and a summary within the 25-word limit. The nondeterminism experiment showed that temperature 0.7 produced greater tag variation than temperature 0.0, although temperature 0.0 did not eliminate variation completely.

The interactive model client followed the bullet-only instructions from AGENT.md, recorded token usage after every response, and displayed conversation statistics without altering the stored history. The final self-check passed all implemented verification checks.